

ARRAYS 👍

1. Solution to Middle Function

```
public class Exercise {  
    public static int[] middle(int[] array) {  
        if (array.length <= 2) {  
            return new int[0]; // Return an  
empty array if the input array has 2 or fewer  
elements  
        }  
  
        // Create a new array with a size of  
the input array length minus 2  
        int[] middleArray = new  
int[array.length - 2];  
  
        // Copy the elements from the input  
array, excluding the first and last elements  
        int index = 1;  
        while (index < array.length - 1) {  
            middleArray[index - 1] =  
array[index];  
            index++;  
        }  
  
        return middleArray;  
    }  
}
```

Here's a brief explanation of what each part of the code does:

1. **public static int[] middle(int[] array) {**: This line defines the middle function, which takes an integer array as input and returns a new integer array containing all elements except the first and last elements.

2. `if (array.length <= 2) {`: This line checks if the input array has 2 or fewer elements.
3. `return new int[0];`: If the input array has 2 or fewer elements, this line returns an empty integer array.
4. `int[] middleArray = new int[array.length - 2];`: This line creates a new integer array named `middleArray` with a length equal to the input array length minus 2.
5. `int index = 1;`: This line initializes the index variable to 1.
6. `while (index < array.length - 1) {`: This line starts a while loop that iterates from index 1 to index `array.length - 2` of the input array.
7. `middleArray[index - 1] = array[index];`: This line copies the current element from the input array to the `middleArray`, excluding the first and last elements.
8. `index++;`: This line increments the index variable by 1.
9. `return middleArray;`: This line returns the `middleArray`, which contains all elements of the input array except the first and last elements.

Time complexity explanation

In the provided `middle` function, the time complexity is determined by the loop that iterates through the elements of the input array.

1. `public static int[] middle(int[] array) {`: Time complexity is $O(1)$ as it defines the function.
2. `if (array.length <= 2) {`: Time complexity is $O(1)$ as it performs a single comparison.
3. `return new int[0];`: Time complexity is $O(1)$ as it returns an empty array.
4. `int[] middleArray = new int[array.length - 2];`: Time complexity is $O(1)$ as it allocates memory for the new array.

5. `int index = 1;` Time complexity is $O(1)$ as it initializes the index variable.
6. `while (index < array.length - 1) {` The time complexity of the loop itself is $O(n-2)$ because it iterates from index 1 to index `array.length - 2` of the input array. However, the loop's time complexity will be dominated by the operations inside the loop.
7. `middleArray[index - 1] = array[index];` Time complexity is $O(1)$ as it performs a single assignment operation.
8. `index++;` Time complexity is $O(1)$ as it increments the index variable by 1.
9. `return middleArray;` Time complexity is $O(1)$ as it returns the `middleArray`.

In summary, the time complexity of the `middle` function is determined by the loop, which has a time complexity of $O(n-2)$. Since we're looking at the overall time complexity and we usually ignore constants, the time complexity of the `middle` function can be considered $O(n)$, where n is the length of the input array.

2. Solution to 2D Arrays

```
public class Exercise {
    public static int
sumDiagonalElements(int[][] array) {
    int sum = 0;
    int numRows = array.length;

    for (int i = 0; i < numRows; i++) {
        sum += array[i][i];
    }
}
```

```
        return sum;
    }
}
```

Explanations:

1. `public static int sumDiagonalElements(int[][] array) {`: This line defines the `sumDiagonalElements` function, which takes a 2D integer array as input and returns the sum of its diagonal elements.
2. `int sum = 0;`: This line initializes the `sum` variable to store the sum of the diagonal elements.
3. `int numRows = array.length;`: This line gets the number of rows in the input 2D array and stores it in the `numRows` variable.
4. `for (int i = 0; i < numRows; i++) {`: This line starts a for loop that iterates through each row of the input 2D array, using the variable `i` as the index.
5. `sum += array[i][i];`: This line adds the diagonal element at position `(i, i)` in the input 2D array to the `sum` variable.
6. `return sum;`: This line returns the sum of the diagonal elements after the loop completes.

Time Complexity Explanation:

1. `public static int sumDiagonalElements(int[][] array) {`: Time complexity is $O(1)$ as it defines the function.
2. `int sum = 0;`: Time complexity is $O(1)$ as it initializes the sum variable.
3. `int numRows = array.length;`: Time complexity is $O(1)$ as it assigns the number of rows in the input 2D array to the `numRows` variable.
4. `for (int i = 0; i < numRows; i++) {`: The time complexity of the loop itself is $O(n)$ because it iterates through each row of the input 2D array, where `n` is the number of rows.

The loop's time complexity will be determined by the operations inside the loop.

5. `sum += array[i][i];` Time complexity is $O(1)$ as it performs a single addition and assignment operation.
6. `return sum;` Time complexity is $O(1)$ as it returns the sum variable.

In summary, the time complexity of the `sumDiagonalElements` function is determined by the for loop, which has a time complexity of $O(n)$, where n is the number of rows in the input 2D array. Since the operations inside the loop also have a time complexity of $O(1)$, the overall time complexity of the `sumDiagonalElements` function is $O(n)$.

3. Solution to Best Score

```
import java.util.Arrays;
import java.util.Collections;
public class Exercise {
    public static int[] findTopTwoScores(int[]
array) {
    int firstHighest = Integer.MIN_VALUE;
    int secondHighest = Integer.MIN_VALUE;

    for (int score : array) {
        if (score > firstHighest) {
            secondHighest = firstHighest;
            firstHighest = score;
        } else if (score > secondHighest &&
score < firstHighest) {
            secondHighest = score;
        }
    }
}
```

```
        return new int[]{firstHighest,  
        secondHighest};  
    }  
}
```

Explanation:

1. `public static int[] findTopTwoScores(int[] array)`
{: This line defines the `findTopTwoScores` function, which takes an integer array as input and returns an integer array containing the first and second highest scores.
2. `int firstHighest = Integer.MIN_VALUE;`: This line initializes the `firstHighest` variable with the minimum possible integer value. This variable will be used to store the highest score in the input array.
3. `int secondHighest = Integer.MIN_VALUE;`: This line initializes the `secondHighest` variable with the minimum possible integer value. This variable will be used to store the second-highest score in the input array.
4. `for (int score : array) {`: This line starts a for-each loop that iterates through each element in the input array, using the variable `score` to store the current element.
5. `if (score > firstHighest) {`: This line checks if the current `score` is greater than the current value of `firstHighest`.
6. `secondHighest = firstHighest;`: If the condition in line 5 is true, this line assigns the current value of `firstHighest` to `secondHighest`.
7. `firstHighest = score;`: If the condition in line 5 is true, this line assigns the current `score` value to `firstHighest`.
8. `} else if (score > secondHighest && score < firstHighest) {`: This line checks if the current `score` is greater than the current value of `secondHighest` and less than the current value of `firstHighest`.

9. `secondHighest = score;` If the condition in line 8 is true, this line assigns the current `score` value to `secondHighest`.
10. `}` This line closes the if-else statement started in line 5.
11. `}` This line closes the for-each loop started in line 4.
12. `return new int[]{firstHighest, secondHighest};`
This line returns an integer array containing the values of `firstHighest` and `secondHighest`.

Time complexity explanation:

1. `public static int[] findTopTwoScores(int[] array)`
{: Time complexity is $O(1)$ as it defines the function.
2. `int firstHighest = Integer.MIN_VALUE;` Time complexity is $O(1)$ as it initializes the `firstHighest` variable.
3. `int secondHighest = Integer.MIN_VALUE;` Time complexity is $O(1)$ as it initializes the `secondHighest` variable.
4. `for (int score : array) {` The time complexity of the loop itself is $O(n)$ because it iterates through each element in the input array, where n is the number of elements. The loop's time complexity will be determined by the operations inside the loop.
5. `if (score > firstHighest) {` Time complexity is $O(1)$ as it performs a single comparison.
6. `secondHighest = firstHighest;` Time complexity is $O(1)$ as it performs a single assignment operation.
7. `firstHighest = score;` Time complexity is $O(1)$ as it performs a single assignment operation.
8. `} else if (score > secondHighest && score < firstHighest) {` Time complexity is $O(1)$ as it performs up to two comparisons.
9. `secondHighest = score;` Time complexity is $O(1)$ as it performs a single assignment operation.
10. `}` Time complexity is $O(1)$ as it closes the if-else statement.

11. `}`: Time complexity is $O(1)$ as it closes the for-each loop.
12. `return new int[]{firstHighest, secondHighest};`
Time complexity is $O(1)$ as it returns an integer array containing two elements.

In summary, the time complexity of the `findTopTwoScores` function is determined by the for-each loop, which has a time complexity of $O(n)$, where n is the number of elements in the input array. Since the operations inside the loop also have a time complexity of $O(1)$, the overall time complexity of the `findTopTwoScores` function is $O(n)$.

4. Solution to Missing Number

```
public class Exercise {  
    public static int  
    findMissingNumberInArray(int[] array) {  
        int n = array.length + 1;  
        int expectedSum = (n * (n + 1)) / 2;  
        int actualSum = 0;  
  
        for (int number : array) {  
            actualSum += number;  
        }  
  
        return expectedSum - actualSum;  
    }  
}
```

Explanation:

1. `public static int findMissingNumberInArray(int[] array) {`: This line defines the `findMissingNumberInArray`

function, which takes an integer array as input and returns an integer representing the missing number.

2. `int n = array.length + 1;` This line calculates the value of `n`, which is the length of the input array plus 1, and stores it in the variable `n`. This represents the total count of integers in the range 1 to `n`.
3. `int expectedSum = (n * (n + 1)) / 2;` This line calculates the sum of the first `n` natural numbers using the formula $(n * (n + 1)) / 2$, and stores the result in the variable `expectedSum`.
4. `int actualSum = 0;` This line initializes the `actualSum` variable with the value 0. This variable will be used to store the sum of the elements in the input array.
5. `for (int number : array) {` This line starts a for-each loop that iterates through each element in the input array, using the variable `number` to store the current element.
6. `actualSum += number;` This line adds the current `number` to the `actualSum` variable, incrementally calculating the sum of the elements in the input array.
7. `}` This line closes the for-each loop started in line 5.
8. `return expectedSum - actualSum;` This line calculates the difference between the `expectedSum` and the `actualSum`, which represents the missing number, and returns it as the result of the function.

Time complexity explanation:

1. `public static int findMissingNumberInArray(int[] array) {` Time complexity is $O(1)$ as it defines the function.
2. `int n = array.length + 1;` Time complexity is $O(1)$ as it calculates `n` based on the array length.
3. `int expectedSum = (n * (n + 1)) / 2;` Time complexity is $O(1)$ as it calculates the expected sum using the formula.

4. `int actualSum = 0;`: Time complexity is $O(1)$ as it initializes the actualSum variable.
5. `for (int number : array) {`: The time complexity of the loop itself is $O(n)$ because it iterates through each element in the input array, where n is the number of elements. The loop's time complexity will be determined by the operations inside the loop.
6. `actualSum += number;`: Time complexity is $O(1)$ as it performs a single addition operation.
7. `}`: Time complexity is $O(1)$ as it closes the for-each loop.
8. `return expectedSum - actualSum;`: Time complexity is $O(1)$ as it calculates the difference between the expected sum and the actual sum and returns the result.

In summary, the time complexity of the `findMissingNumberInArray` function is determined by the for-each loop, which has a time complexity of $O(n)$, where n is the number of elements in the input array. Since the operations inside the loop also have a time complexity of $O(1)$, the overall time complexity of the `findMissingNumberInArray` function is $O(n)$.

5. Solution to Duplicate Number

```
public class Exercise {  
    public static int[] removeDuplicates(int[]  
array) {  
        int n = array.length;  
        int[] uniqueArray = new int[n];  
        int index = 0;
```

```

        for (int i = 0; i < n; i++) {
            boolean isDuplicate = false;
            for (int j = i + 1; j < n; j++) {
                if (array[i] == array[j]) {
                    isDuplicate = true;
                    break;
                }
            }
            if (!isDuplicate) {
                uniqueArray[index++] =
array[i];
            }
        }

        return Arrays.copyOf(uniqueArray,
index);
    }
}

```

Explanation:

1. **public static int[] removeDuplicates(int[] array)**
{: This line defines the **removeDuplicates** function, which takes an integer array as input and returns an integer array as output.
2. **int n = array.length;** This line initializes a variable **n** to the length of the input array.
3. **int[] uniqueArray = new int[n];** This line initializes an integer array **uniqueArray** with the same size as the input array.
4. **int index = 0;** This line initializes an index variable to keep track of the next available position in **uniqueArray**.
5. **for (int i = 0; i < n; i++)** {: This line starts a for loop that iterates through each element in the input array.

6. `boolean isDuplicate = false;` This line initializes a boolean variable `isDuplicate` to false to check if the current element is a duplicate.
7. `for (int j = i + 1; j < n; j++) {` This line starts a nested for loop that iterates through all elements in the input array after the current element.
8. `if (array[i] == array[j]) {` This line checks if the current element is equal to any of the remaining elements in the array.
9. `isDuplicate = true;` This line sets the `isDuplicate` variable to true if a duplicate is found.
10. `break;` This line exits the nested for loop if a duplicate is found.
11. `if (!isDuplicate) {` This line checks if the current element is unique.
12. `uniqueArray[index++] = array[i];` This line adds the unique element to `uniqueArray` and increments the index variable.
13. `return Arrays.copyOf(uniqueArray, index);` This line returns a new array containing the unique elements from `uniqueArray` using the `Arrays.copyOf` method with the length of `index`.

Time Complexity Explanation:

1. `int n = array.length;` This line retrieves the length of the input array, which has a time complexity of $O(1)$.
2. `int[] uniqueArray = new int[n];` This line initializes a new array with the same size as the input array, which has a time complexity of $O(n)$.
3. `int index = 0;` This line initializes an integer variable, which has a time complexity of $O(1)$.

4. `for (int i = 0; i < n; i++) {`: This line starts a loop that iterates through each element in the input array, which has a time complexity of $O(n)$.
5. `boolean isDuplicate = false;`: This line initializes a boolean variable, which has a time complexity of $O(1)$.
6. `for (int j = i + 1; j < n; j++) {`: This line starts a nested loop that iterates through all elements in the input array after the current element, which has a time complexity of $O(n)$.
7. `if (array[i] == array[j]) {`: This line checks if the current element is equal to any of the remaining elements in the array, which has a time complexity of $O(1)$.
8. `isDuplicate = true;`: This line sets the `isDuplicate` variable to true if a duplicate is found, which has a time complexity of $O(1)$.
9. `break;`: This line exits the nested loop if a duplicate is found, which has a time complexity of $O(1)$.
10. `if (!isDuplicate) {`: This line checks if the current element is unique, which has a time complexity of $O(1)$.
11. `uniqueArray[index++] = array[i];`: This line adds the unique element to `uniqueArray` and increments the index variable, which has a time complexity of $O(1)$.
12. `return Arrays.copyOf(uniqueArray, index);`: This line returns a new array containing the unique elements from `uniqueArray` using the `Arrays.copyOf` method with the length of `index`, which has a time complexity of $O(n)$.

Therefore, the overall time complexity of the `removeDuplicates` function is $O(n^2)$.

6. Solution to Remove Duplicates from Sorted Array - LeetCode 26

```
public class Exercise {  
    public int removeDuplicates(int[] nums) {  
        if (nums.length == 0) {  
            return 0;  
        }  
  
        int i = 0;  
  
        for (int j = 1; j < nums.length; j++) {  
            if (nums[j] != nums[i]) {  
                i++;  
                nums[i] = nums[j];  
            }  
        }  
  
        return i + 1;  
    }  
}
```

Explanation:

1. **public int removeDuplicates(int[] nums) {**
Defines the **removeDuplicates** method that takes an input array called **nums** and returns an integer value representing the new length of the array after removing duplicates.
2. **if (nums.length == 0) {**
Checks if the input array is empty (length is 0).
3. **return 0;**
If it's empty, the method returns 0, indicating that there are no elements in the array.
4. **}**
Closes the if block.

5. `int i = 0;`
Initializes a pointer `i` to 0, which will keep track of the last non-duplicate element found in the array.
6. `for (int j = 1; j < nums.length; j++) {`
Initializes a for loop with another pointer `j`, starting from 1, and iterates until the end of the array. The purpose of `j` is to iterate through the array and compare elements with the element pointed to by `i`.
7. `if (nums[j] != nums[i]) {`
Checks if the element at index `j` is not equal to the element at index `i`. If the elements are not equal, it means that the element at `j` is a unique (non-duplicate) element.
8. `i++;`
If the elements at `i` and `j` are not equal, we increment `i` by 1. This prepares the next position to store the unique element.
9. `nums[i] = nums[j];`
This line updates the element at index `i` with the element at index `j`. This operation effectively moves the unique element to its correct position in the updated array, removing any duplicates encountered so far.
10. `}`
Closes the if statement.
11. `}`
Closes the for loop.
12. `return i + 1;`
Returns the new length of the array after removing duplicates. Since the pointer `i` is zero-based, we return `i + 1` to get the correct length.
13. `}`
Closes the `removeDuplicates` method.

Time complexity Explanation:

1. `public int removeDuplicates(int[] nums) {` - No time complexity associated.
2. `if (nums.length == 0) {` - This condition check has a constant time complexity of $O(1)$.
3. `return 0;` - No time complexity associated.
4. `}` - No time complexity associated.
5. `int i = 0;` - This initialization has a constant time complexity of $O(1)$.
6. `for (int j = 1; j < nums.length; j++) {` - This for loop runs $(n - 1)$ times, where n is the length of the input array. The time complexity inside the loop will determine the overall time complexity of the loop.
7. `if (nums[j] != nums[i]) {` - This condition check inside the for loop has a constant time complexity of $O(1)$.
8. `i++;` - This increment operation inside the if statement has a constant time complexity of $O(1)$.
9. `nums[i] = nums[j];` - This assignment operation inside the if statement has a constant time complexity of $O(1)$.
10. `}` - No time complexity associated.
11. `}` - No time complexity associated.
12. `return i + 1;` - This addition operation and return statement have a constant time complexity of $O(1)$.
13. `}` - No time complexity associated.

The for loop in line 7 is the dominant factor that determines the time complexity of this algorithm. Since the loop runs $(n - 1)$ times and all the operations inside the loop have a constant time complexity of $O(1)$, the overall time complexity of the `removeDuplicates` method is $O(n)$, where n is the length of the input array.

7. Solution to Best Time to Buy and Sell Stock - LeetCode 121

```
public class Exercise {  
    public int maxProfit(int[] prices) {  
        int minPrice = Integer.MAX_VALUE;  
        int maxProfit = 0;  
  
        for (int price : prices) {  
            if (price < minPrice) {  
                minPrice = price;  
            } else if (price - minPrice >  
maxProfit) {  
                maxProfit = price - minPrice;  
            }  
        }  
  
        return maxProfit;  
    }  
}
```

In this solution, we iterate through the **prices** array using a for-each loop. We maintain two variables: **minPrice** to keep track of the lowest price encountered so far, and **maxProfit** to store the maximum profit achieved. As we iterate through the array, we check if the current price is less than the current **minPrice**. If so, we update the **minPrice** with the current price. If the current price is not less than **minPrice**, we check if the profit made by selling at the current price (i.e., **price - minPrice**) is greater than the current **maxProfit**. If so, we update the **maxProfit** with the new profit. Finally, we return **maxProfit** as the maximum profit that can be achieved from the given stock prices.

Time complexity:

1. **public int maxProfit(int[] prices) {** - No time complexity associated.

2. `int minPrice = Integer.MAX_VALUE;` - This initialization has a constant time complexity of $O(1)$.
 3. `int maxProfit = 0;` - This initialization has a constant time complexity of $O(1)$.
 4. `for (int price : prices) {` - This for loop iterates through the `prices` array once, which has a length of n . The time complexity inside the loop will determine the overall time complexity of the loop.
 5. `if (price < minPrice) {` - This condition check inside the for loop has a constant time complexity of $O(1)$.
 6. `minPrice = price;` - This assignment operation inside the if statement has a constant time complexity of $O(1)$.
 7. `} else if (price - minPrice > maxProfit) {` - This condition check inside the for loop has a constant time complexity of $O(1)$.
 8. `maxProfit = price - minPrice;` - This subtraction and assignment operation inside the else-if statement have a constant time complexity of $O(1)$.
 9. `}` - No time complexity associated.
 10. `}` - No time complexity associated.
 11. `return maxProfit;` - This return statement has a constant time complexity of $O(1)$.
 12. `}` - No time complexity associated.

 13. The for loop in line 4 is the dominant factor that determines the time complexity of this algorithm. Since the loop runs n times (where n is the length of the input array) and all the operations inside the loop have a constant time complexity of $O(1)$, the overall time complexity of the `maxProfit` method is $O(n)$.
-

8. Solution to Two Sum - LeetCode 1

```
public class Exercise {  
    public int[] twoSum(int[] nums, int target)  
    {  
        for (int i = 0; i < nums.length; i++) {  
            for (int j = i + 1; j <  
nums.length; j++) {  
                if (nums[i] + nums[j] ==  
target) {  
                    return new int[] { i, j };  
                }  
            }  
        }  
        throw new IllegalArgumentException("No  
two sum solution found");  
    }  
}
```

1. **class Exercise {**
Defines a class called **Solution** containing the method for solving the problem.
2. **public int[] twoSum(int[] nums, int target) {**
Defines the **twoSum** method that takes an input array called **nums** and an integer **target**. The method returns an integer array containing the indices of the two numbers that add up to the target.
3. **for (int i = 0; i < nums.length; i++) {**
Initializes an outer loop with the index variable **i**, starting from 0, and iterates until the end of the **nums** array. This loop is responsible for selecting the first number in the pair.
4. **for (int j = i + 1; j < nums.length; j++) {**
Initializes an inner loop with the index variable **j**, starting from **i + 1**, and iterates until the end of the **nums** array. This loop is

responsible for selecting the second number in the pair, ensuring that we do not use the same element twice.

5. `if (nums[i] + nums[j] == target) {`

Checks if the sum of the numbers at indices `i` and `j` is equal to the target. If the condition is true, we have found the two numbers that add up to the target.

6. `return new int[] { i, j };`

If the sum of the numbers at indices `i` and `j` is equal to the target, we return an integer array containing the indices `i` and `j`.

7. `}`

Closes the if statement.

8. `}`

Closes the inner loop.

9. `}`

Closes the outer loop.

10. `throw new IllegalArgumentException("No two sum solution found");`

If the loops complete without finding a solution, we throw an exception. According to the problem constraints, this should never happen, as each input will have exactly one solution.

11. `}`

Closes the `twoSum` method.

12. `}`

Closes the `Exercise` class.

This solution uses two nested loops to find the pair of numbers that add up to the target, resulting in a time complexity of $O(n^2)$, where n is the length of the input array. It does not use any library or data structure. However, this solution is less efficient than the HashMap solution with a time complexity of $O(n)$. We have not covered HashMap yet that is why I have used this solution.
